

Notes Explore Module 1

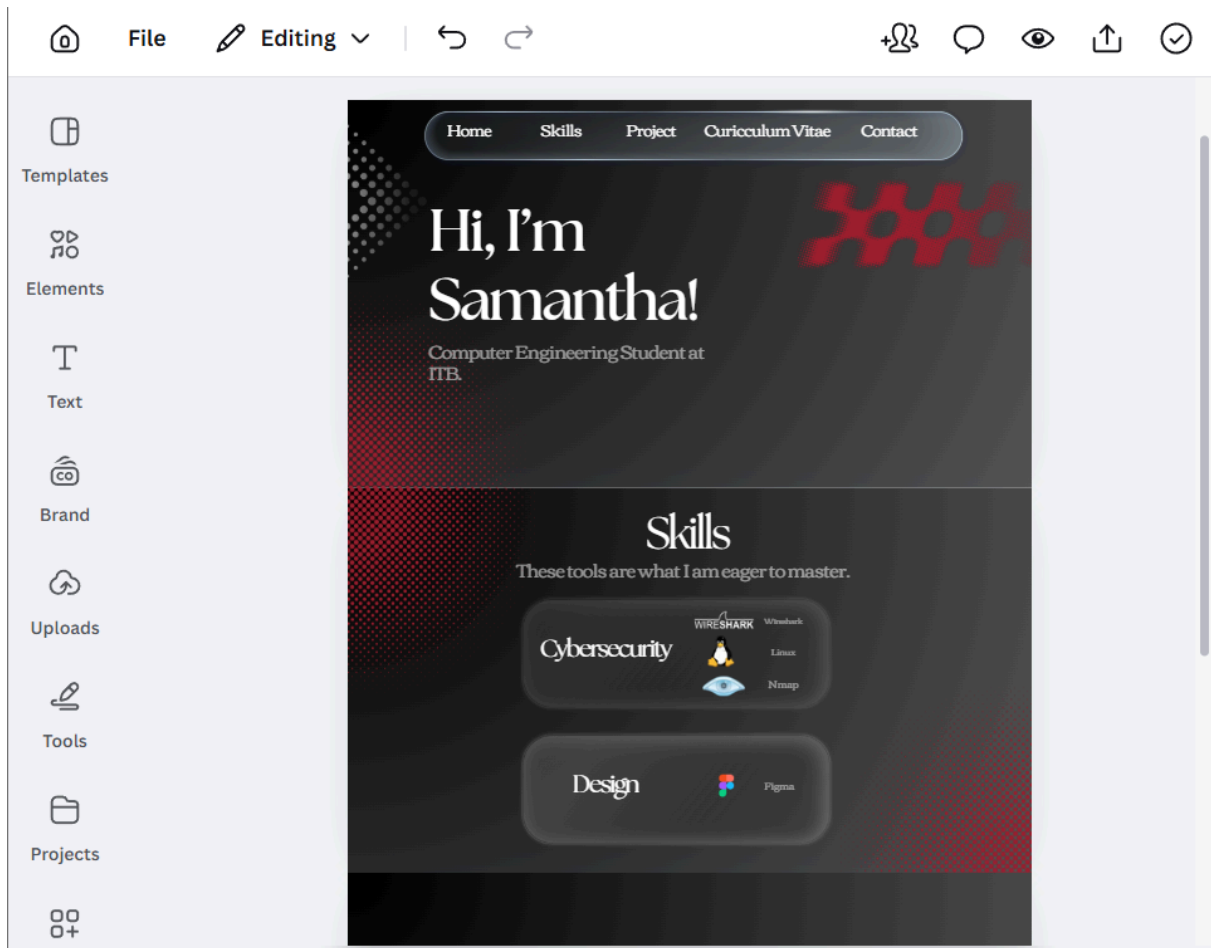
↗ Class	<u>GDGOC</u>
📅 Date	July 3, 2026
☰ Tags	Review

Project Overview

This project is a personal portfolio website built using HTML and CSS. The website includes a hero section, navigation bar, tech skills section, projects section, CV section, and contact footer.

Design Overview

This is the design of the web that I planned to make. However, I made changes (a lot) since I'm still unfamiliar with CSS and HTML. I will try to update the design as time goes by.



Resources

Below are resources that I used to learn and create the web.

HTML basics

A website is usually divided into this basic main sections:

```
<!DOCTYPE html>
<html>
<head>
  <title>Page Title</title>
</head>
<body>
  <h1>Heading 1</h1>
  <div class="class name">
    <p>Paragraph</p>
```

```
    </div>
    <h2>Heading 2</h2>
</body>
</html>
```

- <!DOCTYPE html>: tells the browser to use HTML5 for interpreting
- <html>: root element
- <head>: basic information of the web
- <body> : content of the web
- <h1> : large heading
- <p> : paragraph
- <div> : container inside body

How to :

```
<!--Insert Image-->


<!--Hyperlink-->
<a href="the url or other html files" target=_blank title
="Text that appears when you hover">
    <!-- target can be _blank (new tab) _self (recent tab)
</a>

<!--meta tag-->
<meta name="viewport" content="width=device-width, initial-
scale=1-0">
<meta charset="UTF-8">
<meta http-equiv="refresh" content="30">
```

https://youtu.be/HD13eq_Pmp8?si=FqkF3dTekp6XG45o

Flexbox

Several terms:

- Parent ← the container of elements

- Child ← the elements inside the container
- Main Axis ← main direction of the flex container
- Cross Axis ← opposite direction of the main axis
- `flex-direction` ← controls the direction of the flex items (main axis)
 - row : items go from left to right
 - column : items go from top to bottom
 - row-reverse: : items go from right to left
 - column-revers: items go from bottom to top
- `display : flex` ← to enable flexbox
- `justify-content` ← controls alignment on the main axis.
 - flex-start : items move to the start
 - center : items move to the center
 - flex-end: items move to the end
 - space-between : equal space between items
 - space-around : space around each item
 - space-evenly : equal space around all items
- `align-items` ← controls alignment on the cross axis.
- `gap` : create space between items
- `flex-wrap` ← controls whether items stay in one line or move to the next line (useful for shrinking/enlarge page)
 - nowrap → items stay in one line
 - wrap → items move to next line if there is not enough space
- `flex-grow` ← controls whether an item can grow and take extra space.
 - flex-grow : 0 → item does not grow
 - flex-grow : 1 → item can grow and fill available space (if there are several items, then it grow evenly. You can control the growth speed per items)
- `flex-shrink` ← controls whether an item can shrink when space is limited

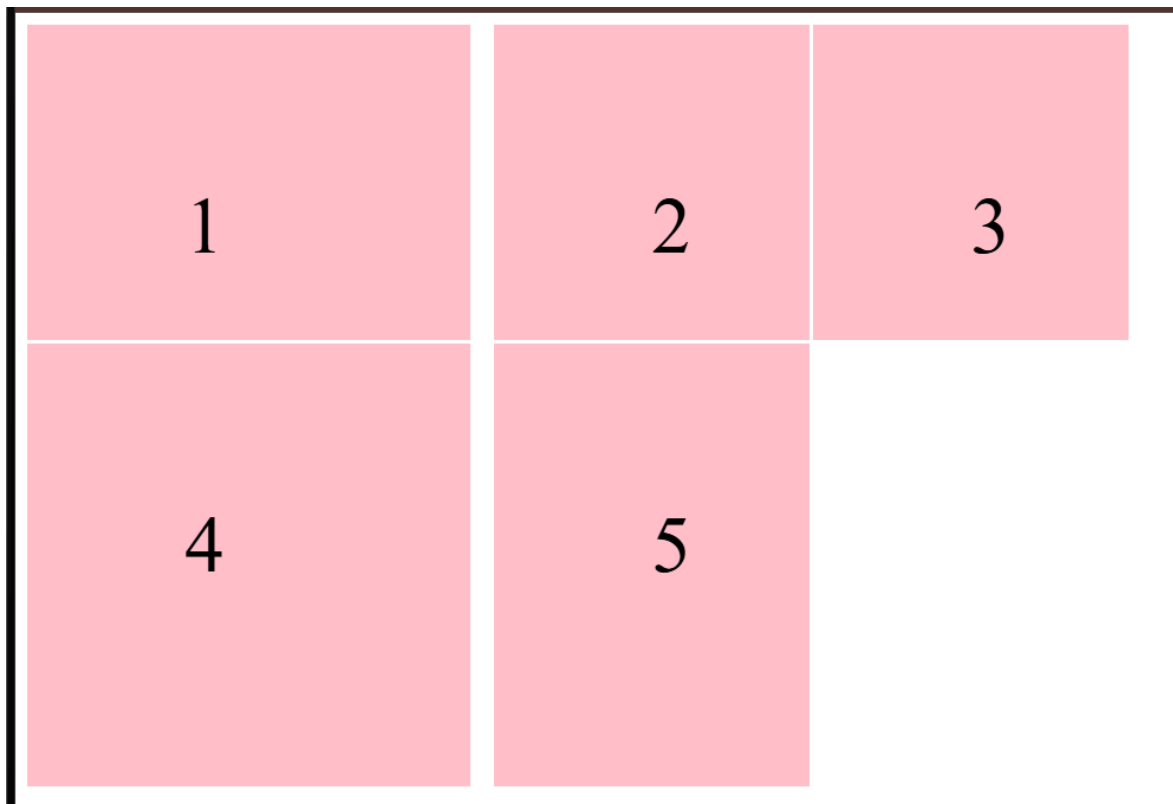
<https://youtu.be/wsTv9y931o8?si=A-UQo0z5zsab5baf>

Grid

Several terms:

- Parent ← the container of elements
- Child ← the elements inside the container
- `display : grid` ← to enable grid
- `grid-template-columns` ← controls grid column size, fixed size
- `grid-template-rows` ← controls grid row size, fixed size

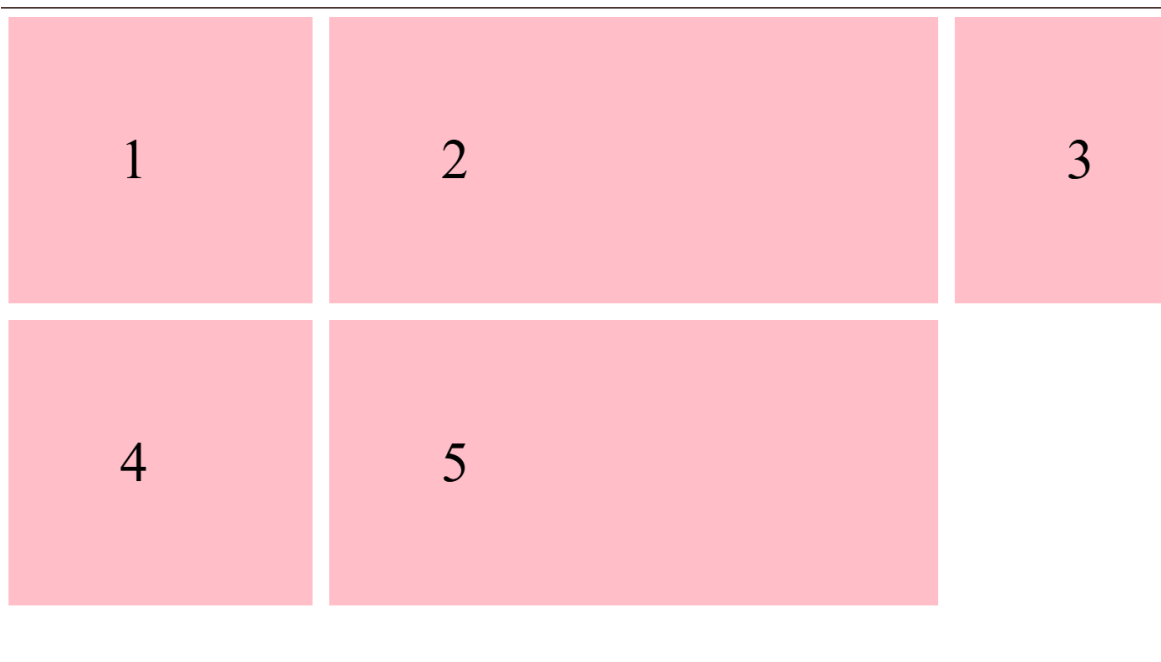
```
grid-template-columns: 300px 200px 200px;  
grid-template-rows: 200px 300px 200px;
```



- `gap` : create space between items
- fraction : controls unfixed size (for responsive grid)

The example below shows that only box 3 is fix-sized even though the page size change.

```
grid-template-columns: 1fr 2fr 200px;  
grid-template-rows: 1fr 1fr 200px;
```



- `justify-items` ← to align items horizontally (start, end center)
- `align-items` ← to align items vertically (start, end center)
- `justify-content` ← align container horizontally
- `align-items` ← align container vertically

Implicit grids:

If we don't know the grid amount, we called it as implicit grids.

- `grid-auto-rows` / `grid-auto-column` ← control automatically generated grid size
- `grid-auto-flow` ← control where the new grid located
 - column : grid created at the new column
 - row : grid created at the new row

Bento grids:

Grids that look like a bento :D (variative grid size)

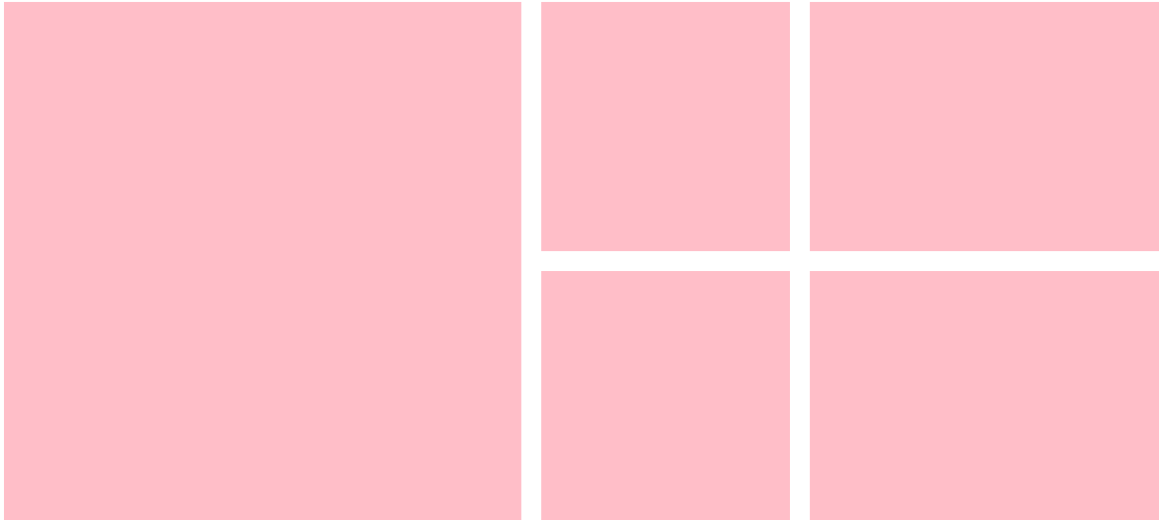
- `grid-template-areas` ← contains information where the boxes are located (matrix-like)

Example:

```
<div class="container">
  <div class="box" style="grid-area: box-1"></div>
  <div class="box" style="grid-area: box-2"></div>
  <div class="box" style="grid-area: box-3"></div>
  <div class="box" style="grid-area: box-4"></div>
  <div class="box" style="grid-area: box-5"></div>
</div>
```

```
.container{
  display: grid;
  grid-template-columns: 200px 200px 200px;
  grid-template-rows: 200px 200px 200px;
  gap: 1em;
  grid-template-areas:
    "box-1 box-1 box-2 box-3"
    "box-1 box-1 box-4 box-5"
}

.box{
  background-color: pink;
}
```



With this, it is easier to make control grids for mobile. We can also automatically create box size without mentioning it one by one using `grid-auto-columns/grid-auto-rows`.

https://youtu.be/JYfiaSKeYhE?si=Qw_j_2ouxxFCVsyV

Additional

- How to hover:

```
.class a:hover{  
  color: ; /*color when hovering*/  
  text-shadow: ;  
}
```